



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY**  
**(AN AUTONOMOUS INSTITUTION)**

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad**

**SDC1-WEEK2-Day4**

**II-I CSE, CSE(AI&ML) AY: 2026-2027**

| <b>SL No</b> | <b>Week</b> | <b>Part</b>   | <b>Detailed Subtopics</b>                                                                                           | <b>Part</b> | <b>Detailed Subtopics</b>        |
|--------------|-------------|---------------|---------------------------------------------------------------------------------------------------------------------|-------------|----------------------------------|
| 1            | Week 2      | <b>Part A</b> | <b>Naive Bayes Classifier: Bayes' Theorem, Conditional Independence, Multinomial Naive Bayes, Numerical Example</b> | Part B      | Naive Bayes Exercise Using SMOTE |

**CONTENTS**

**(Week 2-Day 4 Part A: Naive Bayes Classifier)**

1. Introduction to Naive Bayes Classifiers
2. Core Idea
3. Bayes' Theorem
4. Why Is It Called “Naive”?
5. Step-by-Step Classification Process
6. Types of Naive Bayes Classifiers
7. Numerical Example: Play Tennis Prediction
8. Advantages
9. Limitations
10. Applications

## 1. Introduction to Naive Bayes Classifiers

Naive Bayes is a probabilistic supervised classification algorithm based on Bayes' Theorem, with a naive assumption that features are conditionally independent given the class.

Despite this simplifying assumption, it performs well in practice, especially for:

- Text classification
- Spam filtering
- Sentiment analysis
- Medical diagnosis

## 2. Core Idea

A typical classifier asks:

*“Given these features, what class should I choose?”*

Naive Bayes instead asks the question the other way around:

*“Given this class, how likely are these features?”*

It then uses **Bayes' Theorem** to flip that probability back into an answer for the original question.

## 3. Bayes' Theorem

Bayes' Theorem is the mathematical foundation of the whole algorithm:

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

$C$  = a class (e.g., Spam)  
 $X$  = the observed features  
 $P(C | X)$  = Posterior probability (what we want)  
 $P(X | C)$  = Likelihood  
 $P(C)$  = Prior probability  
 $P(X)$  = Evidence (same for every class)

### For Classification

Since  $P(X)$  is the same value regardless of which class is being tested, it can be dropped when **comparing** classes — it does not change which class scores higher:

$$P(C | X) \propto P(C) \times P(X | C)$$

The predicted class is the one that gives the **maximum** value of this proportional score — this is exactly what “classification” means here: compute the score for every candidate class, and output whichever one is largest. Formally, this is written using **argmax** with the class variable as a subscript (“the class that maximizes this expression”

$$h(x) = \operatorname{argmax}_C P(C) \times P(X|C)$$

*$h(x)$  = the predicted class for input  $x$   
“argmax with subscript  $C$ ” = search every possible class  $C$ , and return whichever one makes  $P(C) \times P(X|C)$  largest*

So instead of computing a true probability, Naive Bayes computes this proportional score for every class and outputs whichever class has the maximum score. No normalization to true probabilities is needed — only the *ranking* of the scores matters for choosing the output.

#### 4. Why Is It Called “Naive”?

Computing  $P(X|C)$  directly — the joint probability of *all* features occurring together — is difficult in practice, since features often influence one another. Naive Bayes simplifies this with a bold assumption:

**Every feature is conditionally independent of every other feature, given the class.**

In other words: once the class is already known, knowing one feature's value gives no extra information about any other feature's value. This lets the joint probability be written as a simple product:

$$P(x_1, x_2, \dots, x_n | C) = P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$$

*where  $x_1, x_2, \dots, x_n$  are the individual features and  $C$  is the class being tested.*

This assumption is called “**naive**” because it is almost never perfectly true in real data — features usually have some relationship with each other. Despite that, treating them as independent still produces a model that ranks classes correctly often enough to be genuinely useful, especially for text data.

#### 5. Step-by-Step Classification Process

##### Step 1: Compute the Prior Probability

$$P(C) = \frac{\text{Number of samples in class } C}{\text{Total samples}}$$

*This reflects how common each class is before looking at any features.*

##### Step 2: Compute the Likelihood for Each Feature

For every feature  $x_i$ , compute  $\mathbf{P(x_i \mid C)}$  — how likely that feature value is, within each class, using the training data.

**Step 3: Multiply the Probabilities**

$$\text{Score}(C) = P(C) \times P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$$

*Computed separately for every candidate class*

**Step 4: Compare and Choose the Maximum**

Whichever class produces the **maximum score** (the *argmax*) is the predicted class — this is the output of the classifier. No normalization to true probabilities is required to make this decision, since only the relative ordering of the scores matters, and taking the maximum only needs a comparison, not exact probability values.

**6. Types of Naive Bayes Classifiers**

| Type                    | Feature Type                                                         | Typical Use Case                                            |
|-------------------------|----------------------------------------------------------------------|-------------------------------------------------------------|
| Gaussian Naive Bayes    | Continuous (numeric) features, assumed normally distributed          | Medical measurements, sensor readings                       |
| Multinomial Naive Bayes | Discrete counts (e.g., word frequencies, as in a Bag-of-Words model) | Text classification, spam filtering                         |
| Bernoulli Naive Bayes   | Binary features (present / absent)                                   | Text classification using word presence only, not frequency |

The worked example in this document applies Naive Bayes to **categorical attributes** (weather conditions) rather than word counts — closer to how *CategoricalNB* is used in practice. The same underlying Bayes' Theorem and independence assumption apply regardless of which feature type is involved; only how each  $\mathbf{P(x_i \mid C)}$  is estimated changes.

**7. Numerical Example: Play Tennis Prediction**

Consider a classic dataset of 14 days, each recording four weather attributes (Outlook, Temperature, Humidity, Wind) and whether a person **played tennis** that day:

| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |

|     |          |      |        |        |     |
|-----|----------|------|--------|--------|-----|
| D4  | Rain     | Mild | High   | Weak   | Yes |
| D5  | Rain     | Cool | Normal | Weak   | Yes |
| D6  | Rain     | Cool | Normal | Strong | No  |
| D7  | Overcast | Cool | Normal | Strong | Yes |
| D8  | Sunny    | Mild | High   | Weak   | No  |
| D9  | Sunny    | Cool | Normal | Weak   | Yes |
| D10 | Rain     | Mild | Normal | Weak   | Yes |
| D11 | Sunny    | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High   | Strong | Yes |
| D13 | Overcast | Hot  | Normal | Weak   | Yes |
| D14 | Rain     | Mild | High   | Strong | No  |

The task: given a **new** day with weather  $X = \{ \text{Outlook} = \text{Sunny}, \text{Temperature} = \text{Cool}, \text{Humidity} = \text{High}, \text{Wind} = \text{Strong} \}$ , predict whether the person will play tennis.

### Step 1: Compute Prior Probabilities for Each Class

Out of 14 days, tennis was played (“Yes”) on 9 days and not played (“No”) on 5 days:

$$P(\text{PlayTennis} = \text{Yes}) = 9/14 \approx 0.643 \quad P(\text{PlayTennis} = \text{No}) = 5/14 \approx 0.357$$

### Step 2: Calculate Likelihoods from the Frequency Table

For every attribute value, count how often it occurs on “Yes” days versus “No” days, then divide by the total number of Yes days (9) or No days (5) to get each likelihood:

| Attribute   | Value    | P(value   Yes) | P(value   No) |
|-------------|----------|----------------|---------------|
| Outlook     | Sunny    | $2/9 = 0.222$  | $3/5 = 0.600$ |
| Outlook     | Overcast | $4/9 = 0.444$  | $0/5 = 0.000$ |
| Outlook     | Rain     | $3/9 = 0.333$  | $2/5 = 0.400$ |
| Temperature | Hot      | $2/9 = 0.222$  | $2/5 = 0.400$ |
| Temperature | Mild     | $4/9 = 0.444$  | $2/5 = 0.400$ |
| Temperature | Cool     | $3/9 = 0.333$  | $1/5 = 0.200$ |
| Humidity    | High     | $3/9 = 0.333$  | $4/5 = 0.800$ |
| Humidity    | Normal   | $6/9 = 0.667$  | $1/5 = 0.200$ |
| Wind        | Weak     | $6/9 = 0.667$  | $2/5 = 0.400$ |

|      |        |               |               |
|------|--------|---------------|---------------|
| Wind | Strong | $3/9 = 0.333$ | $3/5 = 0.600$ |
|------|--------|---------------|---------------|

### Step 3: Apply Bayes' Theorem to Compute a Score for Each Class

For the new day  $X = \{ \text{Sunny, Cool, High, Strong} \}$ , compute the proportional score for each class using

$P(C | X) \propto P(C) \times \prod P(x_i | C)$ :

$$\begin{aligned} P(\text{Yes} | X) &\propto P(\text{Yes}) \times P(\text{Sunny}|\text{Yes}) \times P(\text{Cool}|\text{Yes}) \times P(\text{High}|\text{Yes}) \times P(\text{Strong}|\text{Yes}) \\ &= (9/14) \times (2/9) \times (3/9) \times (3/9) \times (3/9) \approx 0.0053 \end{aligned}$$

$$\begin{aligned} P(\text{No} | X) &\propto P(\text{No}) \times P(\text{Sunny}|\text{No}) \times P(\text{Cool}|\text{No}) \times P(\text{High}|\text{No}) \times P(\text{Strong}|\text{No}) \\ &= (5/14) \times (3/5) \times (1/5) \times (4/5) \times (3/5) \approx 0.0206 \end{aligned}$$

### Step 4: Predict the Class with the Highest Score

Comparing the two scores:

$$P(\text{No} | X) \approx 0.0206 > P(\text{Yes} | X) \approx 0.0053$$

Since  $P(\text{No} | X)$  is larger, the predicted class is **PlayTennis = No**. The classifier predicts the person will **not** play tennis on a Sunny, Cool, High-humidity, Strong-wind day.

#### What This Example Teaches

- The predicted class is decided purely by comparing two proportional scores (0.0053 vs 0.0206) — neither is a true probability, since the shared  $P(X)$  term was deliberately dropped. Only their relative size matters for the argmax decision.
- This example uses categorical attributes (Outlook, Temperature, Humidity, Wind) rather than word counts, showing that Naive Bayes applies just as directly outside of text classification — the same Bayes' Theorem and independence assumption, only the way each  $P(x_i | C)$  is estimated changes.
- “Humidity = High” and “Wind = Strong” both individually favor “No” more strongly than “Yes” does (0.800 vs 0.333, and 0.600 vs 0.333) — multiplying several such features compounds this, which is why “No” wins by roughly 4x despite a lower prior probability (0.357 vs 0.643 for “Yes”).

## 8. Advantages

- Fast to train and predict, even on large datasets
- Performs well with high-dimensional data, such as thousands of vocabulary words in text classification

- Requires relatively little training data to estimate the parameters it needs
- Handles multi-class classification naturally, not just two classes
- Simple to implement and interpret

## 9. Limitations

- **The independence assumption is usually unrealistic:** real features are often correlated, which can bias the probability estimates (though classification accuracy often still holds up).
- **Zero-frequency problem:** without smoothing, any unseen word/feature value forces a probability of exactly zero, eliminating that class regardless of other evidence — this is why Laplace smoothing is applied in practice.
- **Probability estimates are not always reliable:** Naive Bayes often ranks classes correctly, but the actual numeric probabilities it outputs should not be trusted as precise confidence scores.
- **Gaussian Naive Bayes assumes normally distributed features:** this assumption may not hold for all continuous data.

## 10. Applications

- **Spam Filtering:** Classifying emails as spam or legitimate based on word frequencies — one of the most common real-world uses of Multinomial Naive Bayes.
- **Sentiment Analysis:** Classifying reviews or social media posts as positive, negative, or neutral based on word usage.
- **Document Categorization:** Sorting news articles or support tickets into topic categories.
- **Medical Diagnosis:** Estimating the likelihood of a condition given a set of observed symptoms.
- **Recommendation Systems:** Simple content-based filtering using feature likelihoods.

## Q & A

### 1. What is Naive Bayes?

Answer: Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem, assuming that features are conditionally independent of each other.

## 2. Why is it called “Naive” Bayes?

Answer: It is called naive because it assumes that all features are independent of each other given the class.

## 3. What is Bayes' Theorem?

Answer:  $P(C|X) = [P(X|C) \times P(C)] / P(X)$

$P(C|X)$  = Posterior probability

$P(X|C)$  = Likelihood

$P(C)$  = Prior probability

$P(X)$  = Evidence

## 4. What are the main types of Naive Bayes?

Answer: • Gaussian NB – continuous numerical data

• Multinomial NB – count/frequency data, commonly text

• Bernoulli NB – binary features (0/1)

## 5. What is the role of prior probability in Naive Bayes?

Answer: Prior probability represents the probability of a class before considering the input features. It is calculated from the class distribution in the training data.

## B. Scenario-Based Questions

### 6. A churn dataset contains 90% non-churned and 10% churned customers. Can accuracy alone be used to evaluate Naive Bayes?

Answer: No. Accuracy can be misleading because the model may favor the majority class. Use precision, recall, F1-score, and confusion matrix, especially for the churn class.

### 7. A medical dataset has 95% healthy and 5% diseased patients. The model has 95% accuracy but very low disease recall. Is the model good?

Answer: No. A high accuracy can result from predicting most patients as healthy. Recall for the disease class is important because missing diseased patients is costly.



**8. A Naive Bayes model predicts almost every customer as “Not Churn.” What problem might be causing this?**

Answer: Class imbalance may be causing the model to favor the majority class.

**9. How can SMOTE help in an imbalanced churn dataset?**

Answer: SMOTE creates synthetic minority-class samples, increasing the representation of the minority class and helping the model learn its patterns better.

**10. Should SMOTE be applied before or after train-test splitting?**

Answer: Apply SMOTE only to the training data after train-test splitting. The test data should remain untouched to provide an unbiased evaluation of the model.